

Python Coding Test — CineMax Ticket Booking System (No Functions)

Time limit: 50 minutes | Language: Python 3

Rules

- **No functions.** Do not write `def`. Every part is solved with plain top-level code (variables, arithmetic, conditions, loops).
- **No lists or tuples.** Do not use `[]` or `()` to store collections of data (no list, no tuple, no `.append()`). Use plain variables and `range()` instead.
- **No rounding.** Do not use `round()` or `:.2f` formatting anywhere. Print raw computed values exactly as Python produces them, decimals and all.
- **No nested loops.** A `for/while` inside another `for/while` is not allowed. `if` statements inside a loop are fine and expected.

Scenario

You're building a command-line ticket booking system for **CineMax Cinema**. Work through the parts in order — each one reuses variables/logic from the previous part.

Part 1 — Basic Data Types (5 min)

Create these variables, then print them all in one f-string on a single line: - `cinema_name` (str) → "CineMax Cinema" - `base_price` (float) → 12.5 - `seats_available` (int) → 120 - `is_weekend` (bool) → True - `screen_number` (int), read from user input as text and converted to an int → `input("Enter screen number: ")`

Also print the **type** of each variable using `type()`.

Part 2 — Arithmetic Operations (10 min)

Given:

```
adult_tickets = 3
child_tickets = 2
senior_tickets = 1
base_price = 12.5
```

Calculate and print each of these **without rounding**: - `child_price` = `base_price * 0.6` - `senior_price` = `base_price - (base_price // 4)` (use integer floor division here on purpose) - `subtotal` = sum of all three ticket categories at their respective prices - `service_fee` = `subtotal % 5` (use the modulo operator) - `tax` = `subtotal * 0.0825` - `total` = `subtotal + tax + service_fee` - `average_ticket_price` = `total / (adult_tickets + child_tickets + senior_tickets)`

Print every value with a label. Do not round any of them — the decimals should look messy, that's expected.

Part 3 — Conditions (10 min)

CineMax has **three membership tiers**: "bronze", "silver", "gold", plus non-members ("none").

Given a variable `membership = "silver"` and `ticket_count = 7`, write `if / elif / else` logic (using `and / or` where needed — no nested ifs yet) that prints a discount rate based on **both** membership tier **and** ticket count: - "gold" member → 20% discount regardless of ticket count - "silver" member **and** `ticket_count >= 5` → 15% discount - "silver" member **and** `ticket_count < 5` → 8% discount - "bronze" member **or** `ticket_count >= 10` → 5% discount - anything else → 0% discount

Print the discount rate that applies.

Part 4 — Nested Conditions (10 min)

Now combine **three factors** with genuinely nested `if` statements: `is_weekend`, `time_slot` ("matinee" or "evening"), and `membership` ("gold", "silver", "bronze", or "none").

Use these variables:

```
is_weekend = True
time_slot = "evening"
membership = "gold"
base_price = 12.5
```

Logic (must be nested at least 2 levels deep): - If `is_weekend`: - If `time_slot == "evening"`: - "gold" → 10% surcharge - "silver" or "bronze" → 20% surcharge - else ("none") → 30% surcharge - Else ("matinee"): - "gold" → no change - anything else → 10% surcharge - Else (weekday): - If `time_slot == "evening"`: - "gold" or "silver" → 5% discount - else → no change - Else ("matinee"): - any membership → 25% discount

Print the final adjusted price (unrounded).

Part 5a — for loop (7 min)

No lists, no tuples. Use a `for` loop over `range(8)` to process exactly 8 bookings, one at a time, reading each one from input as you go (no storing them anywhere first):

```
for i in range(8):
    age = int(input("Enter age: "))
    ticket_type = input("Enter ticket type (child/adult/senior): ")
    # ... process immediately, don't save into a list
```

For each booking, in the same loop iteration (no nesting, no functions): - Check whether the age actually matches its declared `ticket_type` using these rules: child = under 12, adult = 12-64, senior = 65+. - Print "MISMATCH" if the age doesn't match its declared type, otherwise print "OK". - Keep a running total of ticket revenue as you go, where child = 7.5, adult = 12.5, senior = 9.0 — print the running total after each booking. - At the end (after the loop), print the final total revenue and how many mismatches were found.

Test data to type in when prompted (in order): 8 child, 45 adult, 67 senior, 12 adult, 5 child, 90 senior, 30 adult, 16 child.

Part 5b — while loop (8 min)

Build a booking loop (no nesting, no functions) starting from:

```
seats_available = 5
tickets_booked = 0
total_revenue = 0
```

- Repeatedly ask: "Enter option (book/exit): "
 - If the user types "book":
 - If `seats_available == 0`, print "Sold out!" and do **not** decrement anything
 - Otherwise, decrement `seats_available` by 1, increment `tickets_booked` by 1, add 12.5 to `total_revenue`, and print the current `seats_available` and `total_revenue`
 - If the user types "exit", print the final `tickets_booked`, `seats_available`, and `total_revenue`, then stop the loop
 - Any other input → print "Invalid option"
-

Grading Breakdown (100 points)

Part	Topic	Points
1	Basic data types	10
2	Arithmetic operations	20
3	Conditions	15
4	Nested conditions	25
5a	for loop	15
5b	while loop	15

Notes for candidates: - No `def` anywhere in your submission. - No `[]` or `()` used to hold collections of data — no lists, no tuples. - No `round()`, no `:.2f`, no manual truncation to “clean up” output — leave the raw decimals. - Only one loop level at a time — a `for/while` may not contain another `for/while`. - If you can’t finish a part, leave a comment describing your intended approach for partial credit.